

CORAL-G Run And Reset Guide

Combined running and reset procedure for the CORAL-G Gazebo/Nav2 proof of concept.

This guide combines the normal run procedure and the proper reset procedure for the CORAL-G Gazebo/Nav2 simulation.

It is intended for teammates or future agents who need to run the current CORAL-G digital twin proof of concept from a clean terminal state.

1. What The Simulation Does

The simulation runs a TurtleBot3 in Gazebo, uses Nav2 for navigation, and runs the CORAL-G digital twin nodes for mission-level reasoning.

Runtime loop:

```
Gazebo TurtleBot3
-> /scan and /odom
-> CORAL-G robot/environment/twin/debris/planner nodes
-> /coral_g/goal_pose
-> Nav2 /navigate_to_pose
-> /cmd_vel
-> Gazebo robot moves
```

CORAL-G decides where the robot should go. Nav2 decides how to drive there safely.

2. Expected Demo Behavior

The PoC uses six logical trash targets inside the small map:

```
( 0.75, -0.75)
(-0.75, -0.75)
( 0.75, -1.75)
(-0.75, -1.75)
( 0.75, -2.75)
(-0.75, -2.75)
```

Expected route behavior:

1. Robot starts near base (0, 0).
2. CORAL-G chooses the highest-utility remaining trash cell.
3. Nav2 drives the robot to that goal.
4. On success, CORAL-G marks that logical cell as collected.
5. The collected cell disappears from the debris map.
6. Storage increases by 0.25.
7. Fuel decreases by 0.08.

8. After two collected cells, storage reaches 0.5.
9. CORAL-G switches to `return_to_base`.
10. Robot returns near (0, 0).
11. Fuel resets to 1.0; storage resets to 0.0.
12. Robot continues with the remaining trash cells.

Trash targets are logical digital-twin cells, not physical Gazebo objects.

3. Normal Startup From Closed Terminals

Use three terminals for the normal run, plus one optional monitor terminal.

Terminal 1: Start Docker And Gazebo

Open WSL:

```
cd /home/c2irr10/turtlebot3_ws
docker run --rm -it --name turtlebot3_container --net=host -e DISPLAY=$DISPLAY \
-v /mnt/wslg/.X11-unix:/tmp/.X11-unix \
-v /home/c2irr10/turtlebot3_ws:/ws \
--user $(id -u):$(id -g) turtlebot3_ws bash
```

Inside Docker:

```
cd /ws
source /opt/ros/jazzy/setup.bash
source /opt/turtlebot3_ws/install/setup.bash
export TURTLEBOT3_MODEL=burger
colcon build --packages-select my_tb3_world coral_g_interfaces coral_g
source install/setup.bash
ros2 launch my_tb3_world new_world.launch.py
```

Expected:

- ? Gazebo opens.
- ? TurtleBot3 appears in the world.
- ? Keep this terminal running.

Terminal 2: Start Nav2

Open a second WSL terminal:

```
docker exec -it turtlebot3_container bash
```

Inside Docker:

```
cd /ws
source /opt/ros/jazzy/setup.bash
source /opt/turtlebot3_ws/install/setup.bash
source install/setup.bash
```

```
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True map:="/ws/
Simulation Files/map.yaml"
```

Expected:

- ? RViz opens.
- ? Nav2 starts.
- ? Keep this terminal running.

Terminal 3: Start CORAL-G

Open a third WSL terminal:

```
docker exec -it turtlebot3_container bash
```

Inside Docker:

```
cd /ws
source /opt/ros/jazzy/setup.bash
source /opt/turtlebot3_ws/install/setup.bash
source install/setup.bash
ros2 launch coral_g coral_g_demo.launch.py
```

Important:

- ? `coral_g_demo.launch.py` starts `initial_pose_publisher`.
- ? That node publishes `/initialpose` automatically.
- ? You normally do not need to paste the long `/initialpose` command anymore.

Expected:

- ? CORAL-G nodes start.
- ? Nav2 receives initial pose.
- ? Robot begins navigating between trash cells.
- ? Logs show reports such as:

`demo report received: run_id=coral_g_demo total_collected=...`

4. Optional Monitor Terminal

Open another WSL terminal:

```
docker exec -it turtlebot3_container bash
```

Inside Docker:

```
cd /ws
source /opt/ros/jazzy/setup.bash
```

```
source /opt/turtlebot3_ws/install/setup.bash
source install/setup.bash
```

Navigation Status

```
ros2 topic echo /coral_g/navigation_status
```

Expected pattern:

```
received
active
succeeded
```

Fuel, Storage, And Pose

```
ros2 topic echo /coral_g/robot_state --once --full-length
```

Look for:

```
fuel_level
storage_fill
at_base
pose
navigation_status
```

Current CORAL-G Decision

```
ros2 topic echo /coral_g/next_cell_goal --once --full-length
```

Possible modes:

```
cleanup_cell
return_to_base
idle
```

Remaining Trash Cells

```
ros2 topic echo /coral_g/debris_density_map --once --full-length
```

The number of `density_cells` should decrease as the robot collects logical trash cells.

Full Report

```
ros2 topic echo /report --once --full-length
```

Useful fields:

```
collection_stats.total
robot_summary
next_cell_goal
heatmap_cells
```

5. Proper Reset Procedure

Do not use the Gazebo reset button for this PoC.

The TurtleBot is spawned by ROS launch. Gazebo reset can desync the robot entity, /odom, Nav2, AMCL, or transforms.

Use this shutdown order:

1. Stop CORAL-G
2. Stop Nav2
3. Stop Gazebo / Docker container
4. Restart Gazebo -> Nav2 -> CORAL-G

Step 1: Stop CORAL-G

In the CORAL-G terminal:

```
Ctrl+C
```

Optional check:

```
ros2 node list | grep -E  
'twin_input|material_belief|digital_twin|debris_simulation|robot_state|field_planner|mission_planner|goal_navigation|collection_report|demo_control|initial_pose'
```

Expected: no output.

Step 2: Stop Nav2

In the Nav2 terminal:

```
Ctrl+C
```

Wait for Nav2 and RViz to close.

Step 3: Stop Gazebo

In the Gazebo terminal:

```
Ctrl+C
```

If still inside Docker, type:

```
exit
```

Step 4: Restart Cleanly

Start again in this order:

Terminal 1: Gazebo
Terminal 2: Nav2
Terminal 3: CORAL-G

CORAL-G will publish the initial pose automatically when launched.

6. Resetting Only CORAL-G Logic

If Gazebo and Nav2 are still healthy and you only want to restart mission logic:

1. Stop CORAL-G with `Ctrl+C`.
2. Relaunch:

```
ros2 launch coral_g coral_g_demo.launch.py
```

This resets:

- ? collection history,
- ? material belief,
- ? simulated fuel/storage,
- ? current mission progress.

It does not reset the Gazebo robot pose. If the robot is far from base, CORAL-G starts from the current `/odom` pose.

For a truly fresh run from the start position, restart Gazebo and Nav2 too.

7. If Automatic Initial Pose Does Not Work

If Nav2 logs:

```
AMCL cannot publish a pose or update the transform. Please set the initial pose
```

then CORAL-G may have started before Nav2/AMCL subscribed to `/initialpose`.

Fix:

1. Stop CORAL-G.
2. Confirm Nav2 is running.
3. Launch CORAL-G again.

Manual fallback:

```
ros2 topic pub --once /initialpose geometry_msgs/msg/PoseWithCovarianceStamped "{header: {
frame_id: map}, pose: {pose: {position: {x: 0.0, y: 0.0, z: 0.0}, orientation: {z: 0.0, w: 1.
0}}, covariance: [0.25, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.25, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0685]}}"
```

8. Troubleshooting

Nav2 Action Server Missing

```
ros2 action info /navigate_to_pose
```

Expected:

```
Action servers: 1
  /bt_navigator
```

If action servers is 0, Nav2 is not ready.

CORAL-G Publishes Goals But Robot Does Not Move

Check:

```
ros2 topic echo /coral_g/navigation_status
ros2 topic echo /coral_g/goal_pose --once
ros2 action info /navigate_to_pose
```

Also inspect the Nav2 terminal for:

```
Failed to create plan
Goal is occupied
Start outside bounds
Timed out waiting for transform
```

Manual Nav2 Test

Stop CORAL-G, then run:

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose "{pose: {header: {
frame_id: map}, pose: {position: {x: 0.75, y: -0.75, z: 0.0}, orientation: {w: 1.0}}}}"
```

If the manual goal works but CORAL-G goals fail, inspect:

- ? `/coral_g/goal_pose` stamp,
- ? duplicate goal suppression,
- ? mission goal locking.

Verify CORAL-G Contracts

```
ros2 run coral_g contract_check
```

Expected:

```
CORAL-G contract check passed for 9 JSON contracts.
```

9. Shutdown Summary

Use this order:

1. Ctrl+C CORAL-G
2. Ctrl+C Nav2
3. Ctrl+C Gazebo
4. Exit optional monitor shells

The Docker container uses `--rm`, so it is removed automatically when the original Docker run exits.